

AI Test Agent v3.0

Fresh Architecture: Playwright MCP Server + GitHub Copilot CLI

Document Version: 3.0 — Complete Redesign **Replaces:** v2.0 Sequential Pipeline, v2.0 Multi-Agent Plan **Core Shift:** From "AI generates code blindly" → "AI drives browser in real time"

Table of Contents

1. [Why This Changes Everything](#)
 2. [How Playwright MCP + Copilot CLI Work Together](#)
 3. [Full System Architecture](#)
 4. [Core Components — Detailed Spec](#)
 5. [The Agent Loop — Step by Step](#)
 6. [Copilot Prompt Design for MCP Tool Calling](#)
 7. [Two Execution Modes](#)
 8. [Action Log to Code Generation](#)
 9. [Memory Layer — Redesigned for MCP](#)
 10. [Database Schema](#)
 11. [Message Bus and Events](#)
 12. [Use Cases Remapped](#)
 13. [What Gets Deleted from v2.0](#)
 14. [Build Order — Phase by Phase](#)
 15. [File Structure](#)
 16. [What Done Looks Like](#)
-

1. Why This Changes Everything

The Core Problem With v2.0

Every failure in v2.0 traces back to one root cause: **Copilot generates Playwright code without ever seeing the browser.**

v2.0 Flow:
User Prompt → Copilot imagines the page → Writes selectors →
Execute → Selectors don't exist → Healing → More imagination →
Still fails → Give up

Copilot is writing selectors for a page it has never seen. When you feed it a DOM snapshot, you're showing it a text list of elements — not the live page. It still guesses. That is why you had:

- Hallucinated selectors (#signin-email-input that never existed)
- 2-3 healing attempts on every run
- Shadow DOM failures
- networkidle hangs

What v3.0 Does Instead

v3.0 Flow:
User Prompt → Copilot decides first action →
MCP executes it on real browser → Browser returns what actually happened →
Copilot sees real result → Decides next action →
MCP executes → Loop until complete →
Convert recorded actions to test code → Push

Copilot never guesses. Every decision is grounded in what the browser actually returned.

The Fundamental Difference

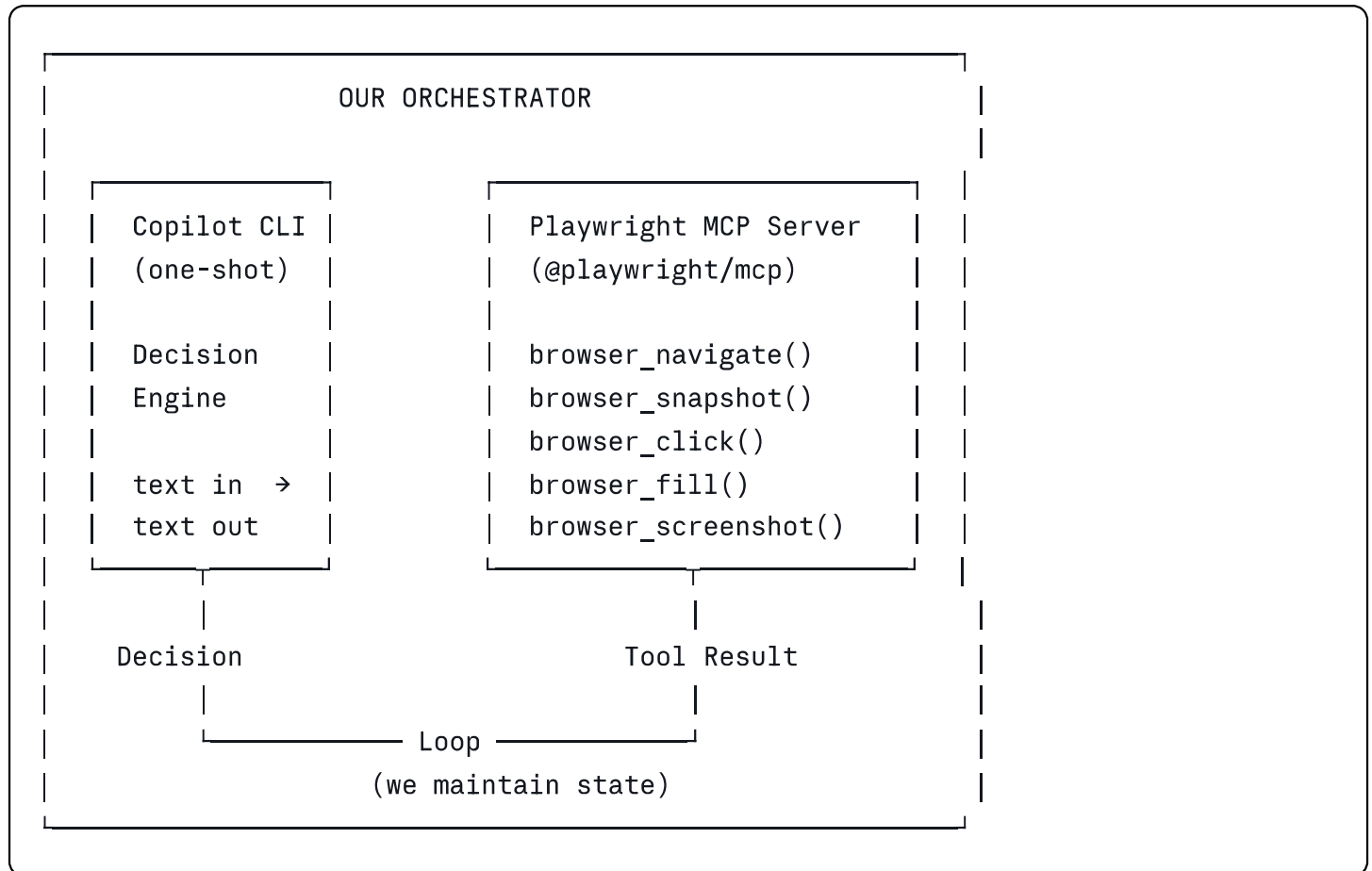
Dimension	v2.0	v3.0
Copilot sees browser	Never	Every step
Selector source	AI imagination	Real accessibility tree
Failure mode	Wrong selectors	Rare — AI adapts in real time
Healing attempts	2-3 per run	0 (self-correcting loop)
First-run pass rate	~40%	~85%+
Code quality	Variable	Generated from proven actions

2. How Playwright MCP + Copilot CLI Work Together

The Integration Problem

`gh copilot` CLI is a **one-shot process**: text in, text out, process exits. MCP requires a **persistent bidirectional connection**. These two facts appear to be incompatible.

The solution: **we build the persistence layer ourselves.**



Our orchestrator is the glue:

- Maintains conversation state across Copilot calls
- Maintains browser state via MCP
- Passes browser snapshots to each Copilot call
- Passes Copilot decisions to MCP
- Loops until the task is done

What Playwright MCP Server Provides

The `@playwright/mcp` package runs as a local subprocess that speaks **JSON-RPC over stdio**:

```
bash
```

```
# Start the server
npx @playwright/mcp --stdio

# Example tool call (JSON-RPC)
→ {"jsonrpc":"2.0","id":1,"method":"tools/call",
    "params":{"name":"browser_navigate",
        "arguments":{"url":"https://app.com"}}}

# Server response
← {"jsonrpc":"2.0","id":1,
    "result":{"content":[
        {"type":"text","text":"Navigated to https://app.com"},
        {"type":"text","text":"Title: App Login"}
    ]}}
```

Available tools:

Tool	What It Returns
browser_navigate	Navigation status, page title
browser_snapshot	Full accessibility tree of current page
browser_click	Click result, updated page state
browser_fill	Fill result
browser_type	Type result
browser_press_key	Key press result
browser_select_option	Selection result
browser_screenshot	Base64 screenshot
browser_evaluate	JavaScript execution result
browser_wait_for	Wait completion status
browser_network_requests	All network requests made
browser_close	Close confirmation

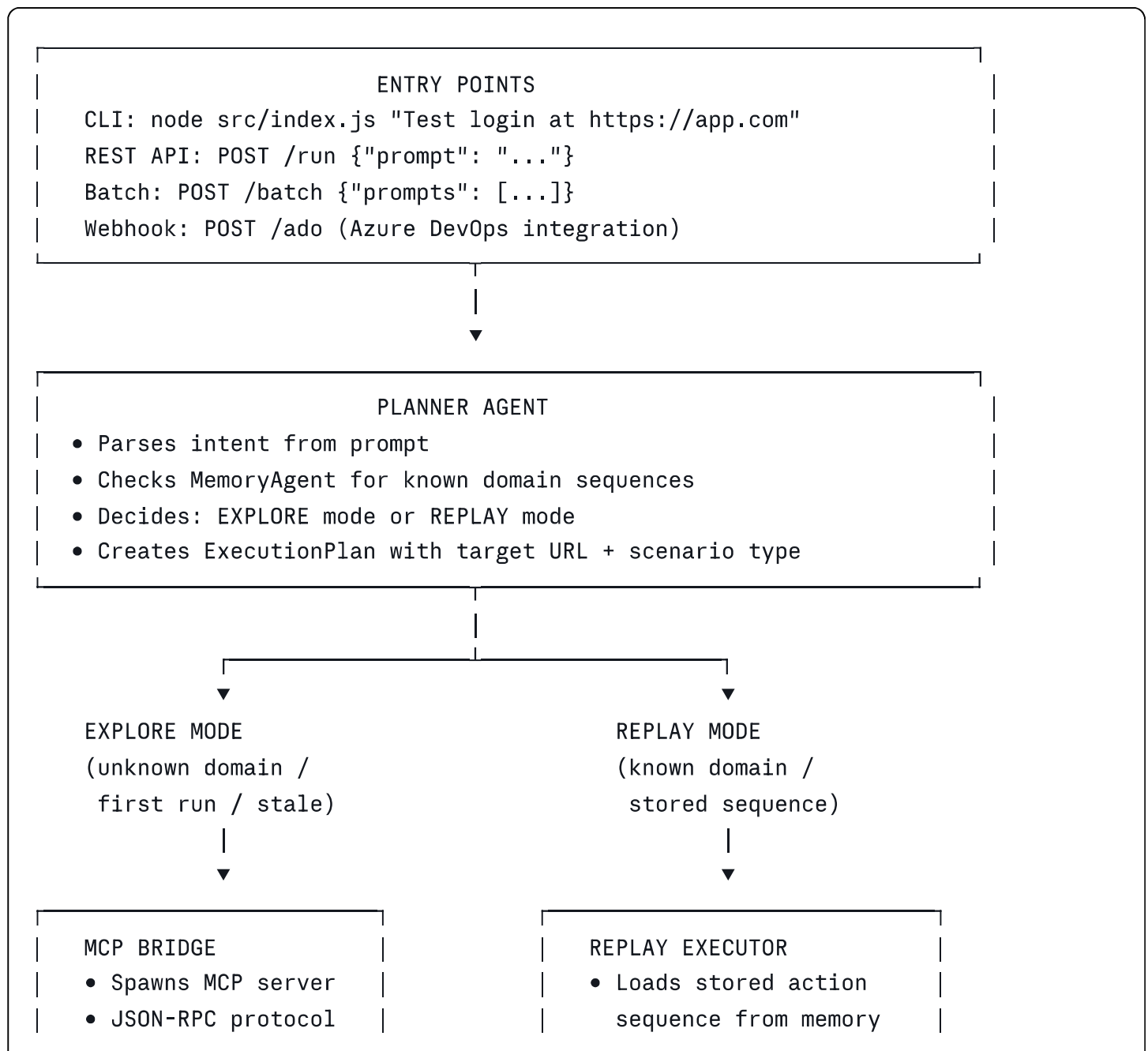
What Each Copilot Call Receives

Every time we call Copilot, we give it:

1. The original test intent (user's prompt)
2. The full list of available MCP tools
3. The current browser accessibility snapshot
4. The history of actions taken so far
5. The result of the last action
6. Instructions to output ONE tool call decision as JSON

Copilot outputs a single structured JSON decision. We execute it. We loop.

3. Full System Architecture



- `callTool()` API

- Executes via MCPBridge
- Falls back to EXPLORE if any step fails

AGENT LOOP

- Builds context prompt for Copilot
- Calls Copilot CLI
- Parses tool decision
- Calls MCPBridge
- Records action
- Checks completion
- Max 25 steps

ACTION RECORDER

Records every: tool called | args used | result received |
accessibility state before and after | screenshot paths

CODE GENERATOR

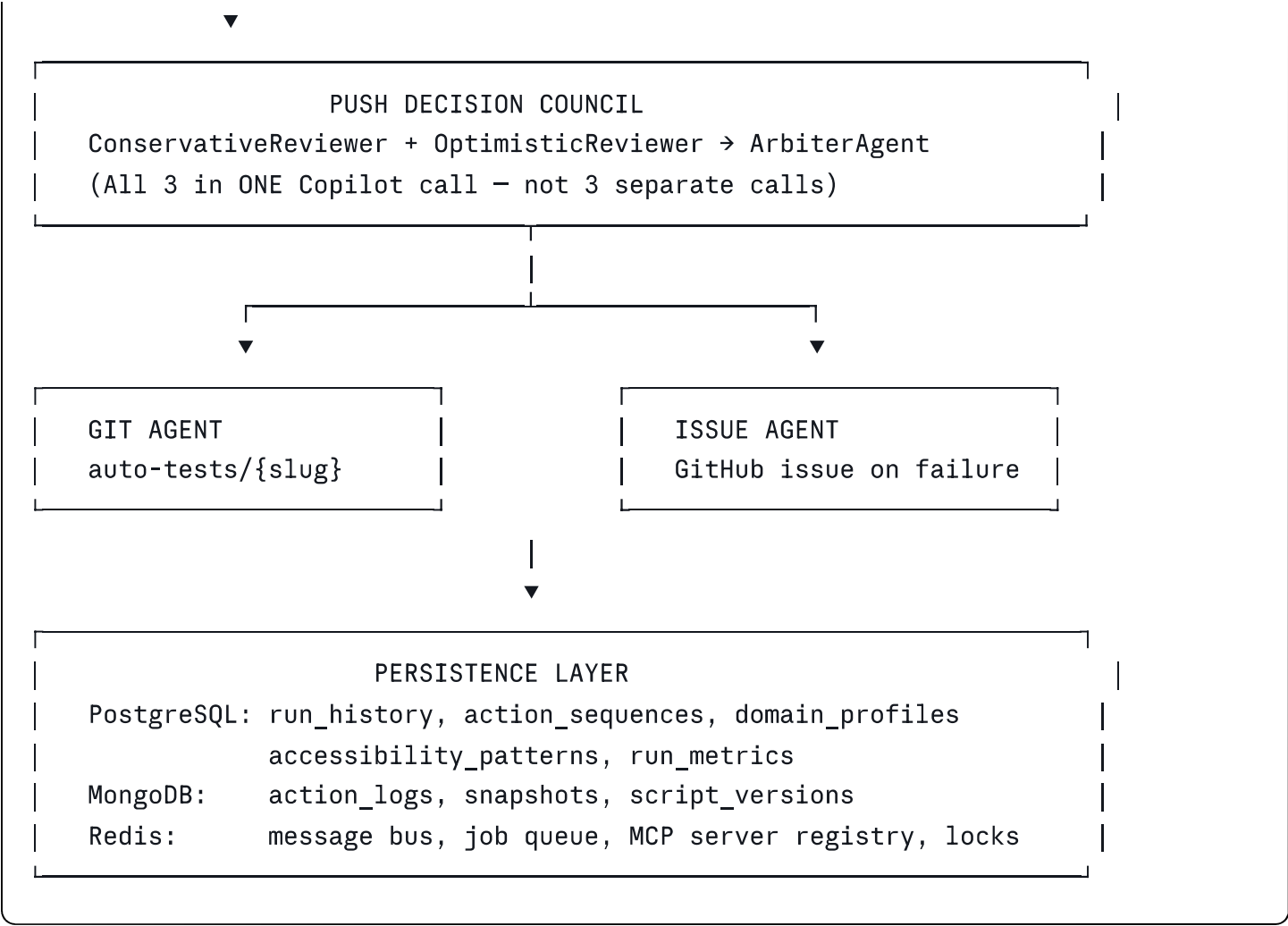
- Takes complete action log
- Calls Copilot: "Convert this browser session to Playwright code"
- Outputs clean `.spec.js` file

STATIC ANALYZER

- Syntax check
- Playwright import
- Assertion presence
- Security scan
- (No selector check --
actions already proved
against live browser)

MEMORY AGENT

- Saves action sequence
- Updates domain profile
- Stores accessibility
patterns (not selectors)
- Writes run history



4. Core Components — Detailed Spec

4.1 MCPBridge

File: `src/core/MCPBridge.js`

Responsibility: Manages the Playwright MCP server subprocess and exposes a clean async API.

javascript

```

class MCPBridge {
  constructor() {
    this.process = null;      // MCP server subprocess
    this.pendingCalls = new Map(); // requestId → {resolve, reject}
    this.messageBuffer = '';  // JSON-RPC message buffer
    this.messageId = 0;
    this.tools = [];          // Discovered tool list
  }

  // Start the MCP server subprocess
  async start() {
    this.process = spawn('npx', ['@playwright/mcp', '--stdio'], {
      stdio: ['pipe', 'pipe', 'pipe']
    });

    // Buffer stdout for complete JSON messages
    this.process.stdout.on('data', (data) => {
      this.messageBuffer += data.toString();
      this.processBuffer();
    });

    // Initialize: discover available tools
    this.tools = await this.listTools();
  }

  // Call any MCP tool by name
  async callTool(toolName, args = {}) {
    const id = ++this.messageId;
    return new Promise((resolve, reject) => {
      this.pendingCalls.set(id, { resolve, reject });

      const message = JSON.stringify({
        jsonrpc: '2.0',
        id,
        method: 'tools/call',
        params: { name: toolName, arguments: args }
      }) + '\n';

      this.process.stdin.write(message);

      // Timeout per tool call
      setTimeout(() => {
        if (this.pendingCalls.has(id)) {

```



```

        this.pendingCalls.delete(id);
        reject(new Error(`MCP tool ${toolName} timed out after 30s`));
    }
    }, 30000);
});
}

// Get accessibility snapshot of current page
async getSnapshot() {
    const result = await this.callTool('browser_snapshot');
    return result.content[0].text;
}

// Navigate and return snapshot
async navigate(url) {
    await this.callTool('browser_navigate', { url });
    return await this.getSnapshot();
}

// Expose all tool definitions for Copilot context
getToolDefinitions() {
    return this.tools.map(t => ({
        name: t.name,
        description: t.description,
        parameters: t.inputSchema
    }));
}

async stop() {
    if (this.process) {
        this.process.kill();
        this.process = null;
    }
}
}

```

Critical design decisions:

- One MCPBridge instance per run (not shared across runs)
- Timeout on every tool call — never hangs
- Message buffering for incomplete JSON-RPC responses
- Clean stop() called in finally block of every run

4.2 AgentLoop

File: `src/agents/AgentLoop.js`

Responsibility: The core intelligence loop. Alternates between asking Copilot "what to do" and executing MCP tool calls.

```
javascript
```

```

class AgentLoop {
  constructor(mcpBridge, actionRecorder, llmClient) {
    this.mcp = mcpBridge;
    this.recorder = actionRecorder;
    this.llm = llmClient;
    this.maxSteps = 25;      // Safety limit
    this.stepCount = 0;
  }

  async run(plan) {
    const { targetUrl, intent, knownPatterns } = plan;

    // Step 1: Navigate to start URL
    const initialSnapshot = await this.mcp.navigate(targetUrl);
    this.recorder.recordAction('browser_navigate', { url: targetUrl }, initialSnapshot);

    // Step 2: Agent loop
    let currentSnapshot = initialSnapshot;
    let isComplete = false;

    while (!isComplete && this.stepCount < this.maxSteps) {
      this.stepCount++;

      // Ask Copilot: given what you see, what is the next action?
      const decision = await this.askCopilotForNextAction({
        intent,
        currentSnapshot,
        actionHistory: this.recorder.getHistory(),
        availableTools: this.mcp.getToolDefinitions(),
        knownPatterns,
        stepNumber: this.stepCount
      });

      // Check if Copilot says task is done
      if (decision.tool === 'COMPLETE') {
        isComplete = true;
        this.recorder.markComplete(decision.reasoning);
        break;
      }

      // Check if Copilot says task is impossible
      if (decision.tool === 'ABORT') {
        throw new Error(`Agent aborted: ${decision.reasoning}`);
      }
    }
  }
}

```

```

    }

    // Execute the chosen tool
    const result = await this.mcp.callTool(decision.tool, decision.args);

    // Get updated snapshot after action
    let updatedSnapshot = currentSnapshot;
    if (decision.tool !== 'browser_screenshot') {
        updatedSnapshot = await this.mcp.getSnapshot();
    }

    // Record the action and result
    this.recorder.recordAction(
        decision.tool,
        decision.args,
        updatedSnapshot,
        result,
        decision.reasoning
    );

    currentSnapshot = updatedSnapshot;
}

if (this.stepCount >= this.maxSteps) {
    throw new Error(`Agent exceeded max steps (${this.maxSteps}). Task may be too
}

return this.recorder.getCompleteLog();
}

async askCopilotForNextAction(context) {
    const prompt = this.buildDecisionPrompt(context);
    const raw = await this.llm.ask(prompt);
    return this.parseDecision(raw);
}

parseDecision(rawResponse) {
    // Extract JSON from response
    const jsonMatch = rawResponse.match(/(?:[\s\S]*?)/);
    if (!jsonMatch) {
        throw new Error('Copilot response did not contain valid JSON decision');
    }
    const decision = JSON.parse(jsonMatch[0]);

```

```
// Validate required fields
if (!decision.tool) throw new Error('Decision missing tool field');
if (!decision.args && decision.tool !== 'COMPLETE') {
    decision.args = {};
}
return decision;
}
}
```

Why max 25 steps?

- Simple tests: 3-8 steps
 - Medium tests (login + verify): 8-15 steps
 - Complex flows: 15-25 steps
 - Beyond 25: something is wrong — abort and report
-

4.3 ActionRecorder

File: `src/core/ActionRecorder.js`

Responsibility: Records every step of the browser session. This is the raw material for code generation and memory storage.

javascript

```

class ActionRecorder {
  constructor(runId, domain, scenarioType) {
    this.runId = runId;
    this.domain = domain;
    this.scenarioType = scenarioType;
    this.actions = [];
    this.startTime = Date.now();
    this.completed = false;
    this.completionReason = null;
  }

  recordAction(tool, args, snapshotAfter, result = null, reasoning = null) {
    this.actions.push({
      step: this.actions.length + 1,
      tool,
      args,
      result: result ? this.extractResultText(result) : null,
      snapshotAfter, // accessibility tree after action
      reasoning, // Copilot's stated reason for this action
      timestamp: Date.now()
    });
  }

  markComplete(reason) {
    this.completed = true;
    this.completionReason = reason;
  }

  getHistory() {
    // Condensed history for Copilot context
    // Full snapshots are too large – summarize after first 3 steps
    return this.actions.map((a, i) => ({
      step: a.step,
      tool: a.tool,
      args: a.args,
      reasoning: a.reasoning,
      // Include full snapshot only for last 3 steps
      snapshot: i >= this.actions.length - 3 ? a.snapshotAfter : '[truncated]'
    }));
  }

  getCompleteLog() {
    return {

```

```

        runId: this.runId,
        domain: this.domain,
        scenarioType: this.scenarioType,
        actions: this.actions,
        totalSteps: this.actions.length,
        durationMs: Date.now() - this.startTime,
        completed: this.completed,
        completionReason: this.completionReason
    };
}

// Distilled sequence for memory storage
getStorableSequence() {
    return this.actions.map(a => ({
        tool: a.tool,
        // Store intent patterns, not exact values
        // "fill email field" not "fill test@test.com"
        intent: this.extractIntent(a),
        accessibilityPattern: this.extractAccessibilityPattern(a)
    }));
}

extractIntent(action) {
    // Convert specific values to reusable patterns
    if (action.tool === 'browser_fill' && action.args.value?.includes('@')) {
        return { tool: action.tool, field: action.args.selector, valueType: 'email' };
    }
    if (action.tool === 'browser_navigate') {
        return { tool: action.tool, urlPattern: new URL(action.args.url).pathname };
    }
    return { tool: action.tool, args: action.args };
}

extractResultText(result) {
    if (!result?.content) return null;
    return result.content
        .filter(c => c.type === 'text')
        .map(c => c.text)
        .join('\n')
        .substring(0, 500);
}
}

```

4.4 CodeGenerator

File: `src/agents/CodeGenerator.js`

Responsibility: Takes a complete action log and generates clean Playwright test code via Copilot.

```
javascript
```



```

class CodeGenerator {
  async generate(actionLog) {
    const prompt = this.buildCodeGenPrompt(actionLog);
    const raw = await this.llm.ask(prompt);
    const code = this.extractCode(raw);
    this.validateCode(code);
    return code;
  }

  buildCodeGenPrompt(log) {
    const stepsDescription = log.actions.map((a, i) =>
      `Step ${i + 1}: Called ${a.tool}(${JSON.stringify(a.args)})
      Reason: ${a.reasoning}
      Result: ${a.result || 'success'}`
    ).join('\n\n');

    return `

```

You are converting a recorded browser session into a Playwright test script.

```

DOMAIN: ${log.domain}
SCENARIO: ${log.scenarioType}
TOTAL STEPS: ${log.totalSteps}
COMPLETED: ${log.completed} (${log.completionReason})

```

```

RECORDED BROWSER SESSION:
${stepsDescription}

```

```

FINAL PAGE STATE (Accessibility Snapshot):
${log.actions[log.actions.length - 1]?.snapshotAfter || 'N/A'}

```

INSTRUCTIONS:

1. Convert each recorded step into the equivalent Playwright action
2. Use ONLY getByRole(), getByLabel(), getByPlaceholder(), getByText() selectors
3. These selectors come from the accessibility snapshot – they match the real DOM
4. Add expect() assertions after each significant state change
5. Use waitFor() for dynamic elements shown in the snapshots
6. Use CommonJS: const { test, expect } = require('@playwright/test')
7. Wrap all steps in a single test() block
8. Use process.env for any credential values – NEVER hardcode them
9. Output ONLY the code in a single ```javascript code block

IMPORTANT: The actions above were actually executed against the real browser. Every selector and interaction is proven to work. Translate faithfully.

```
`;  
}  
}
```

Why this produces better code than v2.0:

- Every action in the log was actually executed against the real browser
 - Selectors come from real accessibility snapshots, not imagination
 - Copilot is translating, not inventing
 - No healing needed because the actions already worked
-

4.5 ReplayExecutor

File: `src/agents/ReplayExecutor.js`

Responsibility: On known domains, execute stored action sequences directly via MCP without any Copilot involvement.

javascript

```

class ReplayExecutor {
  async execute(storedSequence, mcpBridge) {
    for (const step of storedSequence) {
      try {
        await mcpBridge.callTool(step.tool, step.resolvedArgs);
      } catch (err) {
        // Step failed – fall back to EXPLORE mode
        return {
          success: false,
          failedAtStep: step.step,
          reason: err.message
        };
      }
    }
    return { success: true };
  }

  resolveArgs(storedStep, context) {
    // Replace stored patterns with actual values
    // e.g., { valueType: 'email' } → process.env.TEST_EMAIL
    const args = { ...storedStep.args };
    if (storedStep.valueType === 'email') args.value = process.env.TEST_EMAIL;
    if (storedStep.valueType === 'password') args.value = process.env.TEST_PASSWORD;
    return args;
  }
}

```

The business value: On the 5th run of the same domain, zero Copilot CLI calls are made. The MCP server executes the stored sequence directly. Sub-30-second runs. Zero AI cost.

5. The Agent Loop — Step by Step

Here is exactly what happens for a prompt: "Test login at https://app.com/login"

STEP 0: Setup

- |— MCPBridge.start() - spawns @playwright/mcp --stdio
- |— ActionRecorder created for runId
- |— Tools list fetched from MCP server

STEP 1: Navigate

- |— mcp.navigate("https://app.com/login")
- |— Returns accessibility snapshot:
 - | [region "Login Form"]
 - | [textbox "Email address"]
 - | [textbox "Password" type=password]
 - | [button "Sign In"]
 - | [link "Forgot password?"]
- |— recorder.recordAction("browser_navigate", {url}, snapshot)

STEP 2: Copilot Decision #1

- |— PROMPT TO COPILOT:
 - | "Task: Test login. You see: [accessibility snapshot].
 - | History: [navigate to login page].
 - | What is the single next action?"
- |— COPILOT RESPONSE:
 - | {"tool":"browser_fill","args":{"selector":"[aria-label='Email address']",
 - | "value":"test@example.com"},"reasoning":"Fill email field first"}
- |— Execute: mcp.callTool("browser_fill", {...})

STEP 3: Copilot Decision #2

- |— Snapshot now shows email field filled
- |— COPILOT RESPONSE:
 - | {"tool":"browser_fill","args":{"selector":"[aria-label='Password']",
 - | "value":"testpass123"},"reasoning":"Fill password field"}
- |— Execute: mcp.callTool("browser_fill", {...})

STEP 4: Copilot Decision #3

- |— Both fields filled
- |— COPILOT RESPONSE:
 - | {"tool":"browser_click","args":{"selector":"[role='button'][name='Sign In']"},
 - | "reasoning":"Submit the login form"}
- |— Execute: mcp.callTool("browser_click", {...})

STEP 5: Copilot Decision #4

- |— New snapshot shows dashboard page
- |— URL changed to /dashboard
- |— COPILOT RESPONSE:

```
| {"tool":"COMPLETE","args":{}},  
| "reasoning":"Login successful. Dashboard visible. Task complete."}  
└─ Loop ends.
```

STEP 6: Code Generation

```
└─ ActionLog has 4 steps (navigate + 3 interactions)  
└─ Copilot call: "Convert this to Playwright code"  
└─ Generated code:  
| test('Login flow', async ({ page }) => {  
|   await page.goto('https://app.com/login');  
|   await page.getByLabel('Email address').fill(process.env.TEST_EMAIL);  
|   await page.getByLabel('Password').fill(process.env.TEST_PASSWORD);  
|   await page.getByRole('button', { name: 'Sign In' }).click();  
|   await expect(page).toHaveURL(/dashboard/);  
| });  
└─ Selectors come from real accessibility tree - not guessed
```

TOTAL COPILOT CALLS: 5 (4 decisions + 1 code generation)

TOTAL MCP CALLS: 5 (navigate + 3 interactions + snapshot checks)

HEALING ATTEMPTS: 0

6. Copilot Prompt Design for MCP Tool Calling

This is the most important design decision in the entire system. Getting this prompt wrong means bad tool decisions.

Decision Prompt Template

```
javascript
```

```

buildDecisionPrompt(context) {
  const {
    intent,
    currentSnapshot,
    actionHistory,
    availableTools,
    knownPatterns,
    stepNumber
  } = context;

  // Truncate snapshot for token management
  const snapshotForPrompt = currentSnapshot.length > 3000
    ? currentSnapshot.substring(0, 3000) + '\n[...truncated for length]'
    : currentSnapshot;

  // Condense history to last 5 steps only
  const recentHistory = actionHistory.slice(-5).map(a =>
    `Step ${a.step}: ${a.tool}(${JSON.stringify(a.args)}) → ${a.reasoning}`
  ).join('\n');

  return `
You are a browser test automation agent executing step ${stepNumber}/25.

TASK: ${intent}

AVAILABLE TOOLS:
${availableTools.map(t => `- ${t.name}: ${t.description}`).join('\n')}
Special responses: COMPLETE (task done), ABORT (task impossible)

CURRENT PAGE STATE (Accessibility Snapshot):
${snapshotForPrompt}

RECENT ACTIONS TAKEN:
${recentHistory || 'None yet'}

${knownPatterns.length > 0 ? `KNOWN PATTERNS FOR THIS DOMAIN:\n${knownPatterns.map(p
  RULES:
1. Output ONLY a single JSON object – no explanation outside JSON
2. Choose ONE action that moves toward completing the task
3. Use accessibility selectors from the snapshot (role, label, name)
4. If the task appears complete, use COMPLETE
5. If the task is impossible (element not found after navigating), use ABORT

```

6. Never repeat an action that already succeeded
7. For fill actions, use placeholder or aria-label as the selector

RESPOND WITH THIS JSON FORMAT ONLY:

```
{
  "tool": "tool_name_or_COMPLETE_or_ABORT",
  "args": { "selector": "...", "value": "..." },
  "reasoning": "one sentence why"
}
;
```

Prompt Size Management

Each Copilot call has a token limit. As steps accumulate, context grows. Rules:

- Full snapshot: only for current step
- Action history: last 5 steps only (older steps summarized)
- Tool list: full (never truncate — Copilot needs all options)
- Known patterns: full (critical for domain knowledge)
- Snapshot: hard cap at 3000 chars, truncate beyond

If step count > 15 and snapshot keeps changing: add instruction "Focus only on completing the remaining steps. Previous context is sufficient."

7. Two Execution Modes

EXPLORE Mode (Copilot-driven)

When triggered:

- Domain not in MemoryAgent
- Stored sequence is marked stale
- Replay failed at any step
- User explicitly requests fresh run

Flow: Full agent loop as described above. Every step decided by Copilot. Full action log recorded.

REPLAY Mode (Memory-driven)

When triggered:

- Domain found in `action_sequences` table
- Sequence state is `verified`
- Last successful replay < 7 days ago

Flow:

Load stored sequence from MemoryAgent

↓

Start MCPBridge

↓

Execute each step directly via MCP (no Copilot calls)

↓

If any step fails → immediately switch to EXPLORE mode

↓

If all steps pass → update sequence.last_replayed_at

↓

Code generation (if code is stale or script was changed)

The economic argument for REPLAY mode:

Scenario	Copilot Calls	MCP Calls	Duration
EXPLORE (first run)	8-15	10-20	2-4 min
REPLAY (known domain)	0-1	5-10	15-30 sec
REPLAY with stale code	1	5-10	30-45 sec

After 10 domains have verified sequences, 90% of daily runs are REPLAYs.

8. Action Log to Code Generation

The action log is the bridge between the MCP session and the final test artifact.

Action Log Structure (MongoDB)

javascript


```

{
  _id: ObjectId,
  runId: "uuid",
  domain: "app.com",
  scenarioType: "authentication",
  actions: [
    {
      step: 1,
      tool: "browser_navigate",
      args: { url: "https://app.com/login" },
      result: "Navigated successfully. Title: Login",
      snapshotAfter: "..full accessibility tree..",
      reasoning: "Navigate to the starting page",
      timestamp: 1234567890
    },
    {
      step: 2,
      tool: "browser_fill",
      args: { selector: "[aria-label='Email address']", value: "test@test.com" },
      result: "Filled successfully",
      snapshotAfter: "..snapshot with filled field..",
      reasoning: "Fill email field as first login step",
      timestamp: 1234567891
    }
    // ... more steps
  ],
  totalSteps: 5,
  durationMs: 12000,
  completed: true,
  completionReason: "Dashboard loaded after login",
  copilotCallsUsed: 5,
  codeGenerated: true,
  scriptPath: "tests/generated/app.com/run-uuid.spec.js"
}

```

Code Generation Quality Rules

The prompt to Copilot for code generation enforces:

- `getByRole` / `getByLabel` / `getByPlaceholder` — from the accessibility snapshot
- No CSS selectors with IDs or classes
- No hardcoded credentials
- Assertions after each significant state change

- `waitFor` for elements that appeared dynamically

Credential Handling

Steps that filled passwords or emails store `valueType` not the actual value:

```
javascript
```

```
// Stored in action_sequences (NOT the action_logs)
{ tool: "browser_fill", field: "[aria-label='Email']", valueType: "email" }
{ tool: "browser_fill", field: "[type='password']", valueType: "password" }

// Generated code
await page.getByLabel('Email address').fill(process.env.TEST_EMAIL);
await page.getByLabel('Password').fill(process.env.TEST_PASSWORD);
```

Actual test credentials never appear in MongoDB, PostgreSQL, or generated code.

9. Memory Layer — Redesigned for MCP

The memory model changes fundamentally. v2.0 stored CSS selectors. v3.0 stores **action sequences and accessibility patterns**.

Why Accessibility Patterns, Not Selectors

CSS selectors break when DOM changes. Accessibility patterns are more stable because:

- `getByRole('button', { name: 'Sign In' })` survives CSS class changes
- `getByLabel('Email address')` survives ID changes
- Only breaks if the button text or label changes

When a pattern breaks, the diff is obvious: "button named 'Sign In' → now named 'Log In'". Easy to fix vs "class `btn-primary` → `btn-login`" which gives no semantic hint.

Memory Layer Components

ActionSequenceStore (PostgreSQL) Stores verified sequences per domain + scenario:

```
domain: "app.com"
scenario: "authentication"
sequence: [{tool, intent, accessibilityPattern}...]
successCount: 12
failureCount: 0
state: "verified" // candidate | verified | stale | deprecated
lastReplayedAt: timestamp
```

AccessibilityPatternStore (PostgreSQL) Stores stable element patterns per domain:

```
domain: "app.com"
intent: "login_email_field"
pattern: "getByLabel('Email address')"
alternativePattern: "getByPlaceholder('Email')"
successRate: 0.97
state: "verified"
```

DomainProfile (PostgreSQL) Now includes MCP-specific flags:

```
domain: "app.com"
requiresAuth: true
shadowDom: false
spa: true
avgStepsForLogin: 5
avgCopilotCallsPerRun: 6
avgRunDurationMs: 15000
replaySuccessRate: 0.94
lastExploreAt: timestamp
```

ActionLog (MongoDB) Full raw session data — large, kept 90 days, then archived.

10. Database Schema

PostgreSQL Tables

```
sql
```

```
-- Domain profiles (expanded from v2.0)
```

```
CREATE TABLE domain_profiles (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  domain VARCHAR(255) UNIQUE NOT NULL,  
  shadow_dom BOOLEAN DEFAULT FALSE,  
  auth_required BOOLEAN DEFAULT FALSE,  
  spa BOOLEAN DEFAULT FALSE,  
  volatility VARCHAR(10) DEFAULT 'medium',  
  avg_steps_per_run DECIMAL(5,2) DEFAULT 0,  
  avg_copilot_calls_per_run DECIMAL(5,2) DEFAULT 0,  
  avg_run_duration_ms INTEGER DEFAULT 0,  
  replay_success_rate DECIMAL(5,4) DEFAULT 0,  
  total_explore_runs INTEGER DEFAULT 0,  
  total_replay_runs INTEGER DEFAULT 0,  
  last_explore_at TIMESTAMP,  
  last_run_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);
```

```
-- Action sequences: stored replay programs
```

```
CREATE TABLE action_sequences (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  domain VARCHAR(255) NOT NULL,  
  scenario_type VARCHAR(50) NOT NULL,  
  sequence JSONB NOT NULL,          -- [{tool, intent, accessibilityPattern, args  
  source_run_id VARCHAR(100),      -- which explore run produced this  
  success_count INTEGER DEFAULT 0,  
  failure_count INTEGER DEFAULT 0,  
  state VARCHAR(20) DEFAULT 'candidate', -- candidate | verified | stale | depreca  
  last_replayed_at TIMESTAMP,  
  last_failed_at TIMESTAMP,  
  created_at TIMESTAMP DEFAULT NOW(),  
  UNIQUE(domain, scenario_type)  
);
```

```
-- Accessibility patterns: stable selectors per domain
```

```
CREATE TABLE accessibility_patterns (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  domain VARCHAR(255) NOT NULL,  
  intent VARCHAR(100) NOT NULL,    -- e.g., 'login_email', 'search_input'  
  pattern TEXT NOT NULL,           -- e.g., "getByLabel('Email address')"  
  alternative_pattern TEXT,        -- fallback if primary fails
```

```

    success_count INTEGER DEFAULT 0,
    failure_count INTEGER DEFAULT 0,
    success_rate DECIMAL(5,4) DEFAULT 0,
    state VARCHAR(20) DEFAULT 'candidate',
    last_verified_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(domain, intent, pattern)
);

-- Run history (replaces v2.0 version, adds MCP metrics)
CREATE TABLE run_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    run_id VARCHAR(100) UNIQUE NOT NULL,
    prompt TEXT NOT NULL,
    domain VARCHAR(255),
    scenario_type VARCHAR(50),
    execution_mode VARCHAR(10),          -- explore | replay
    total_steps INTEGER DEFAULT 0,
    copilot_calls INTEGER DEFAULT 0,
    mcp_tool_calls INTEGER DEFAULT 0,
    passed BOOLEAN DEFAULT FALSE,
    healing_attempts INTEGER DEFAULT 0, -- should always be 0 in v3.0
    push_decision VARCHAR(10),
    duration_ms INTEGER,
    cold_start BOOLEAN DEFAULT FALSE,
    replay_fallback_to_explore BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Failure log (simplified – most failures now MCP-level, not selector-level)
CREATE TABLE failure_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    run_id VARCHAR(100) NOT NULL,
    domain VARCHAR(255) NOT NULL,
    scenario_type VARCHAR(50) NOT NULL,
    failed_at_step INTEGER,
    tool_that_failed VARCHAR(100),
    error_type VARCHAR(50),              -- navigation | element_not_found | timeout |
    error_message TEXT,
    snapshot_at_failure TEXT,           -- accessibility snapshot when failure occurred
    copilot_last_decision TEXT,        -- what Copilot decided before failure
    created_at TIMESTAMP DEFAULT NOW()
);

```

```
-- Pattern library (kept from v2.0, now stores accessibility rules)
CREATE TABLE pattern_library (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  pattern_key VARCHAR(200) UNIQUE NOT NULL,
  scope VARCHAR(30) NOT NULL,
  scope_value VARCHAR(255),
  scenario_type VARCHAR(50),
  description TEXT NOT NULL,
  rule TEXT NOT NULL,
  confidence_score DECIMAL(5,4) DEFAULT 0,
  state VARCHAR(20) DEFAULT 'candidate',
  created_at TIMESTAMP DEFAULT NOW(),
  activated_at TIMESTAMP
);

-- Indexes
CREATE INDEX idx_action_sequences_domain ON action_sequences(domain, scenario_type,
CREATE INDEX idx_accessibility_patterns_domain ON accessibility_patterns(domain, int
CREATE INDEX idx_run_history_domain ON run_history(domain, created_at DESC);
CREATE INDEX idx_failure_log_domain ON failure_log(domain, scenario_type, created_at
```

MongoDB Collections

```
javascript
```

```

// Action logs – full session data
{
  collection: "action_logs",
  schema: {
    runId: String,
    domain: String,
    scenarioType: String,
    executionMode: String,      // 'explore' | 'replay'
    actions: [{
      step: Number,
      tool: String,
      args: Object,
      result: String,
      snapshotAfter: String,    // accessibility tree
      reasoning: String,
      timestamp: Date
    }],
    totalSteps: Number,
    durationMs: Number,
    completed: Boolean,
    completionReason: String,
    copilotCallsUsed: Number
  }
}

// Script versions – same as v2.0
{
  collection: "script_versions",
  schema: {
    runId: String,
    domain: String,
    version: Number,
    content: String,
    generatedFromRunId: String, // which action log produced this
    createdAt: Date
  }
}

// Accessibility snapshots – key pages per domain
{
  collection: "page_snapshots",
  schema: {
    domain: String,

```

```
    url: String,  
    snapshotHash: String,  
    accessibilityTree: String,  
    capturedAt: Date,  
    runId: String  
  },  
  indexes: [  
    { domain: 1, capturedAt: -1 }  
  ]  
}
```

11. Message Bus and Events

typescript


```
export const EVENTS = {
  // Planning
  PLAN_REQUESTED:      'plan.requested',
  PLAN_CREATED:        'plan.created',

  // Execution mode decision
  EXPLORE_REQUESTED:   'explore.requested',
  REPLAY_REQUESTED:    'replay.requested',
  REPLAY_FAILED:       'replay.failed',      // triggers fallback to explore

  // MCP lifecycle
  MCP_STARTED:         'mcp.started',
  MCP_STOPPED:         'mcp.stopped',
  MCP_TOOL_CALLED:     'mcp.tool.called',    // for logging
  MCP_TOOL_FAILED:     'mcp.tool.failed',

  // Agent loop
  AGENT_LOOP_STARTED:  'agent.loop.started',
  AGENT_LOOP_STEP:     'agent.loop.step',
  AGENT_LOOP_COMPLETE: 'agent.loop.complete',
  AGENT_LOOP_ABORTED:  'agent.loop.aborted',
  AGENT_LOOP_TIMEOUT:  'agent.loop.timeout',

  // Code generation
  CODE_GEN_REQUESTED:  'codegen.requested',
  CODE_GEN_COMPLETE:   'codegen.complete',
  CODE_GEN_FAILED:     'codegen.failed',

  // Analysis
  STATIC_ANALYSIS_REQUESTED: 'analysis.static.requested',
  STATIC_ANALYSIS_COMPLETE:  'analysis.static.complete',

  // Memory
  SEQUENCE_SAVE_REQUESTED: 'memory.sequence.save',
  PATTERN_SAVE_REQUESTED:  'memory.pattern.save',
  FAILURE_RECORD:          'memory.failure.record',

  // Push decision
  PUSH_REVIEW_REQUESTED:  'push.review.requested',
  PUSH_DECISION_MADE:     'push.decision.made',

  // Output
  PUSH_REQUESTED:        'git.push.requested',
```

```
PUSH_COMPLETE:           'git.push.complete',  
ISSUE_REQUESTED:         'issue.create.requested',  
} as const;
```

12. Use Cases Remapped

UC-001: First Run — Unknown Domain

v2.0 behavior: Generate code blind → fail → heal → push.

v3.0 behavior:

1. PlannerAgent: cold start detected, EXPLORE mode
 2. AgentLoop: navigates, Copilot drives step by step
 3. Agent sees real page, adapts in real time
 4. Code generated from proven actions
 5. Action sequence stored as `candidate`
 6. After 3 successful replays → promoted to `verified`
-

UC-002: Repeat Run — Known Domain

v2.0 behavior: Same as first run. No memory of what worked.

v3.0 behavior:

1. PlannerAgent: `verified` sequence found → REPLAY mode
 2. ReplayExecutor: executes sequence directly via MCP
 3. Zero Copilot calls
 4. Completes in ~20 seconds
-

UC-003: Shadow DOM (MSN, Salesforce)

v2.0 behavior: Hallucinated selectors. Multiple failures.

v3.0 behavior:

1. MCP `browser_snapshot` returns accessibility tree — Shadow DOM is transparent to accessibility APIs
2. Copilot sees: `[button "Personalize"]` — not the Shadow DOM host element

3. Decision: `browser_click` on `[role='button'][name='Personalize']`
4. MCP handles Shadow DOM piercing internally
5. Works first time

Shadow DOM is an MCP-level problem, not a prompt-level problem. MCP solves it.

UC-004: Auth-Gated Pages

v2.0 behavior: Session management was manual, often failed.

v3.0 behavior:

1. PlannerAgent detects `auth_required` in domain profile
 2. AuthSetupSequence runs first (stored replay sequence for login)
 3. MCP maintains browser session state
 4. Main test sequence runs in authenticated context
 5. No separate storageState files needed — MCP browser is already logged in
-

UC-005: Dynamic Content / SPAs

v2.0 behavior: `networkidle` hangs, dynamic selectors fail.

v3.0 behavior:

1. Agent calls `browser_snapshot` after each action
 2. Snapshot reflects actual current DOM state
 3. If element not visible yet: Copilot decision is `browser_wait_for`
 4. After wait: new snapshot shows element appeared
 5. Next decision proceeds with confirmed visible element
-

UC-006: Test Task Is Not Completable (404, Broken Flow)

New use case unique to v3.0:

1. Agent navigates → page shows 404
2. Snapshot shows: `[heading "Page Not Found"]`
3. Copilot decision: `{"tool": "ABORT", "reasoning": "Page not found, cannot test"}`
4. AgentLoop throws controlled error
5. IssueAgent creates GitHub issue: "Page unavailable during test"

6. Run terminates cleanly — no infinite retry

13. What Gets Deleted from v2.0

These components are **removed entirely** in v3.0:

Removed Component	Reason
<code>DOMWorker.js</code>	MCP <code>browser_snapshot</code> replaces custom DOM extraction
<code>HealingWorker.js</code>	Agent loop self-corrects — no post-execution healing needed
<code>SpecialistGeneratorAgent.js</code>	Agent loop replaced code generation from scratch
<code>RuntimeAnalyzerAgent.js</code>	Agent loop observes results in real time — no post-mortem needed
<code>PatternMinerAgent.js</code>	Patterns extracted from action logs, not failure logs
<code>copilot-provider.ts</code> (old)	LlmClient.js replaces it
<code>test-script-prompt.yaml</code>	Code gen prompt is now in CodeGenerator.js
<code>scenario-strategy.ts</code>	Strategy is determined by Copilot at each step, not pre-defined

These components **change significantly**:

Changed Component	What Changes
<code>PlannerAgent.js</code>	Now decides EXPLORE vs REPLAY mode
<code>MemoryAgent.js</code>	Stores sequences + accessibility patterns, not CSS selectors
<code>StaticAnalyzerAgent.js</code>	Simpler — no selector pre-check needed (actions already proven)
<code>PushDecisionCouncil.js</code>	3 reviewers in 1 Copilot call (not 3 calls)
<code>LlmClient.js</code>	Add timeout, rate limit backoff, prompt size management

These components **stay the same**:

Unchanged Component	Why
MessageBus.js	Correct design, keep it
GitAgent.js	Keep (fix branch naming bug from v2.0 review)
IssueAgent.js	Keep
db/init.js	Keep schema, add new v3.0 tables

14. Build Order — Phase by Phase

Phase 1: MCPBridge Foundation (Week 1-2)

Goal: Establish MCP communication layer. Nothing else builds without this.

Week 1:

- Install @playwright/mcp package
- Build MCPBridge.js: spawn server, JSON-RPC protocol, callTool(), getSnapshot()
- Test every MCP tool manually: navigate, snapshot, click, fill, screenshot
- Write unit tests: start bridge → navigate → get snapshot → stop bridge
- Verify accessibility tree output format — this is what Copilot will read

Week 2:

- Build ActionRecorder.js: record actions, getHistory(), getCompleteLog()
- Build basic AgentLoop: hardcode 3 decisions to test the loop mechanics
- Run end-to-end: bridge starts → navigate → 3 manual tool calls → bridge stops → log recorded
- Measure: how large is a typical accessibility snapshot? (impacts Copilot prompt size)

Exit criteria:

- MCPBridge starts, sends tool calls, receives results reliably
- Tool calls complete within timeout
- Snapshot size measured and prompt budget calculated

Phase 2: Agent Loop with Copilot (Week 3-4)

Goal: Copilot makes decisions, MCP executes them. End-to-end for simple tests.

Week 3:

- Build decision prompt template in AgentLoop
- Wire Copilot CLI → decision parsing → MCP tool execution
- Test with simple scenario: "Verify page loads at <https://demo.playwright.dev/todomvc>"
- Expected: navigate → snapshot → Copilot says COMPLETE → done in 2 steps
- Measure: how many Copilot calls for simple vs complex test

Week 4:

- Test authentication scenario: "Test login at <https://demo.playwright.dev>"
- Expected: navigate → fill username → fill password → click login → COMPLETE
- Handle: Copilot makes wrong decision → MCP returns error → next decision corrects it
- Add max steps guard (25 steps)
- Add prompt truncation for large snapshots

Exit criteria:

- Simple tests complete in < 5 Copilot calls
 - Auth tests complete in < 10 Copilot calls
 - Agent handles element-not-found gracefully (Copilot tries alternative)
-

Phase 3: Code Generation + Static Analysis (Week 5-6)

Goal: Action log → clean Playwright test code

Week 5:

- Build `CodeGenerator.js`: action log → Copilot prompt → code extraction
- Test: take recorded action log, generate code, compare to manually written code
- Verify: generated selectors use getByRole/getByLabel (from accessibility tree)
- Verify: no hardcoded credentials

Week 6:

- Simplify `StaticAnalyzerAgent.js` for v3.0 (remove selector pre-check)
- Wire full pipeline: Prompt → Plan → AgentLoop → ActionLog → CodeGen → StaticAnalysis → Push Decision
- Test 5 different domains

- Measure first-run pass rate

Exit criteria:

- Generated code passes static analysis on first attempt
 - Selectors are accessibility-based (no CSS class selectors)
 - Full pipeline runs end-to-end without crashes
-

Phase 4: Memory Layer (Week 7-9)

Goal: Second run on same domain is faster. Third run is instant (REPLAY).

Week 7:

- Build `action_sequences` table and PostgreSQL operations in MemoryAgent
- After successful agent loop: save action sequence as `candidate`
- Build `ReplayExecutor.js`
- Test: run twice on same domain → second run attempts REPLAY

Week 8:

- Promote sequences from candidate → verified after 3 successful replays
- Add fallback: REPLAY fails → switch to EXPLORE, record failure, update sequence state
- Build `accessibility_patterns` table and population logic
- Measure: what is the replay success rate after 10 test runs?

Week 9:

- Build pattern injection: load active patterns from `pattern_library` and inject into AgentLoop decision prompt
- Run 50 test executions across 5 domains
- Measure: `copilot_calls_per_run` in EXPLORE mode decreasing over time?
- Target: 30% reduction in Copilot calls by run 10 vs run 1 for same domain

Exit criteria:

- REPLAY mode works for 3 domains
 - REPLAY success rate > 80%
 - Fallback to EXPLORE works cleanly when REPLAY fails
-

Phase 5: Batch + CI + Observability (Week 10-12)

Goal: Production-ready. Multiple domains. CI integration.

Week 10:

- Batch orchestration: accept array of prompts, run in parallel
- Each domain gets its own MCPBridge instance (no sharing)
- Aggregated batch report

Week 11:

- Fix all v2.0 bugs carried forward:
 - GitAgent branch naming (`auto-tests/{slug}`)
 - Push council 3-in-1 Copilot call
 - ESM/CJS consistency
 - Run timeout handler

Week 12:

- Observability dashboard data: copilot calls per run, replay rate, step counts
- CI webhook endpoint
- Alert: if explore→replay transition rate drops (patterns not learning)

Exit criteria:

- 5-domain batch run completes in parallel
- CI integration working
- Replay rate > 60% of all runs by end of week 12

15. File Structure

```
src/
├─ core/
│   ├── MCPBridge.js           ← NEW: MCP server subprocess manager
│   ├── ActionRecorder.js      ← NEW: Records browser session
│   ├── MessageBus.js          ← KEPT: Redis pub/sub (unchanged)
│   └─ LlmClient.js            ← UPDATED: timeout + rate limit backoff
│
├─ agents/
│   └─ PlannerAgent.js         ← UPDATED: decides EXPLORE vs REPLAY
```



```

|   ├── AgentLoop.js           ← NEW: Copilot-driven MCP execution loop
|   ├── ReplayExecutor.js      ← NEW: Executes stored sequences via MCP
|   ├── CodeGenerator.js       ← NEW: action log → Playwright code
|   ├── StaticAnalyzerAgent.js ← SIMPLIFIED: no selector pre-check
|   ├── MemoryAgent.js         ← UPDATED: sequences + patterns
|   ├── PushDecisionCouncil.js ← UPDATED: 3-in-1 Copilot call
|   ├── GitAgent.js            ← BUG-FIXED: branch naming
|   └── IssueAgent.js          ← KEPT: unchanged
|
|   └── config/
|       ├── PromptLoader.js    ← KEPT
|       └── prompts.yaml       ← UPDATED: new prompt templates
|
|   └── db/
|       └── init.js            ← UPDATED: v3.0 schema, ESM fixed
|
|   └── index.js               ← UPDATED: v3.0 orchestration flow
|
|   tests/
|       └── generated/
|           ├── {domain}/
|           └── {runId}.spec.js
|
|   package.json              ← ADD: @playwright/mcp

```

New Dependency

```

json
{
  "dependencies": {
    "@playwright/mcp": "latest",
    "@playwright/test": "^1.60.0",
    "bullmq": "^5.77.6",
    "dotenv": "^17.4.2",
    "mongodb": "^7.2.0",
    "pg": "^8.21.0",
    "redis": "^5.12.1",
    "uuid": "^14.0.0"
  }
}

```

Only one new package: `@playwright/mcp`. Everything else stays.

16. What Done Looks Like

Run 1 on a New Domain

```
[PlannerAgent]      Cold start: msn.com – EXPLORE mode
[MCPBridge]         Starting @playwright/mcp server...
[MCPBridge]         Server ready. 12 tools available.
[AgentLoop]         Step 1: Navigate to https://msn.com
[AgentLoop]         Step 2: Copilot → browser_click (personalize button)
[AgentLoop]         Step 3: Copilot → browser_snapshot (verify panel opened)
[AgentLoop]         Step 4: Copilot → browser_fill (search for Times of India)
[AgentLoop]         Step 5: Copilot → browser_press_key (Enter)
[AgentLoop]         Step 6: Copilot → browser_click (follow button)
[AgentLoop]         Step 7: Copilot → COMPLETE
[CodeGenerator]     Generating code from 7-step action log...
[StaticAnalyzer]    Score: 10/10 – all checks passed
[MemoryAgent]       Sequence saved as candidate (msn.com / interaction)
[PushCouncil]       APPROVED – clean first run
[GitAgent]          Pushed to auto-tests/msn-personalize-follow
```

Copilot calls: 8 (7 decisions + 1 code gen)

MCP tool calls: 7

Healing attempts: 0

Duration: 3m 20s

Run 5 on the Same Domain (REPLAY)

```
[PlannerAgent]      Known domain: msn.com – REPLAY mode
[MCPBridge]         Starting @playwright/mcp server...
[ReplayExecutor]    Executing 7-step verified sequence...
[ReplayExecutor]    Step 1/7 ✓ Step 2/7 ✓ Step 3/7 ✓
[ReplayExecutor]    Step 4/7 ✓ Step 5/7 ✓ Step 6/7 ✓
[ReplayExecutor]    Step 7/7 ✓ Sequence complete
[MemoryAgent]       Sequence success count: 5 → stability: 100%
[PushCouncil]       APPROVED – verified replay sequence
[GitAgent]          Pushed to auto-tests/msn-personalize-follow
```

Copilot calls: 0

MCP tool calls: 7

Healing attempts: 0
Duration: 28 seconds

That is the system working correctly. The difference between Run 1 and Run 5 is not a better AI model — it's accumulated knowledge executing directly via browser automation, with no AI needed at all.

Document Version: 3.0 Architecture: Playwright MCP + GitHub Copilot CLI Core Innovation: AI drives browser in real time — no blind code generation Next Review: After Phase 2 completion (Week 4)